

lean-poly-mcp

Technical Study Guide for the Interview

One-sentence framing: This project models an MCP tool interface as a polynomial functor, runs a Lean-checked proof pipeline over real Mathlib category theory statements, and treats OpenAI/Anthropic/Aristotle as untrusted generators whose outputs must be verified locally.

Most important honesty point: The system can generate and verify candidate statements, but it does not currently prove novelty. A successful generated theorem is Lean-verified, not automatically new mathematics.

1. Project Map



The app is a demo surface over a typed proof pipeline. The trust boundary is Lean, not the language model or Aristotle.

- **Poly/** formalizes polynomial functors, lenses, sums, products, and composition.
- **Mcp/** exposes tools like `hello`, `check`, and `search` through JSON-RPC over stdio.
- **Oracle/** elaborates candidate Lean code, audits axioms, and checks statement equality.
- **Formalize/** mines Mathlib CategoryTheory declarations and contains the copresheaf bridge.
- **web/** provides the local demo: mining, candidate scoring, theorem proposing, Aristotle polling, and replay-friendly artifacts.

2. Core Claims to Defend

Claim	What it means	What not to overclaim
MCP interface as polynomial functor	Requests are positions; valid responses are directions over each position. Tool registries are coproducts of these interfaces.	This is not a proof about arbitrary network behavior; it is a typed model of the interface shape.
Agents as lenses	A server is a section of the interface. A stateful agent composed with a server becomes a state transition.	No LangGraph is required for this small chain; the agentic behavior is explicit state update.
Lean as trust boundary	External systems can propose statements or proof terms, but accepted results must elaborate, pass an axiom whitelist, and match the requested statement.	Aristotle success emails are not evidence until local verification succeeds.
Mathlib miner	The miner produces real proof-reconstruction tasks from existing CategoryTheory declarations.	An interesting miss is not a new theorem; it is a known theorem not reproved by the cheap local tactic ladder.

3. Polynomial Functor and MCP Interface

The conceptual dictionary is the cleanest technical story. A polynomial functor is a family of response directions over request positions. For an MCP tool interface, a request such as check has exactly one matching response family: an oracle outcome. The type system prevents confusing a search response with a check response inside Lean.

Interview version: I am not merely wrapping a JSON API. I first model the legal request/response surface as a polynomial interface, then flatten it to JSON only at the transport boundary.

Key correspondences:

Polynomial/categorical term	Project meaning
position	a request shape, such as calling check or search
direction over a position	the response type valid for that request
lens/section	an implementation of the server or agent step
coproduct	a registry combining multiple tools
composition	agent step composed with server implementation

Useful equations to remember:

```
Lens p y ~≡ ((i : p.Pos) -> p.Dir i)
Lens (S y^S) y ~≡ (S -> S)
```

Plain English: a server chooses a valid response for every request; a stateful agent interacting with a server is a state transition.

4. Lean Oracle and Soundness Gates



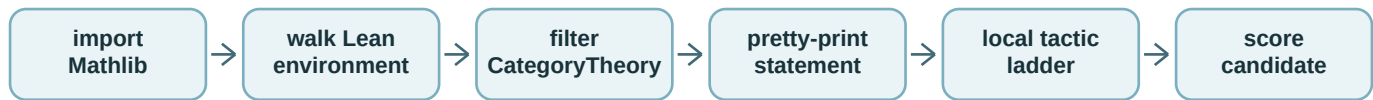
The oracle is intentionally stricter than checking that a process exited successfully.

- **Elaboration:** Lean must accept the term or proof file.
- **Axiom audit:** dependencies must be contained in `propext`, `Classical.choice`, and `Quot.sound`.
- **Statement match:** the proof must prove the requested statement, not a nearby easier statement.
- **Attack examples covered by tests:** sorry, custom axioms, `native_decide` trust axioms, wrong declaration names, and statement mismatch.

Why statement match matters: A model can turn a false or hard prompt into a nearby true statement. The proof can be valid while the formalization is wrong. The oracle rejects this by comparing the accepted theorem statement to the requested one.

5. Mathlib CategoryTheory Miner

The miner is the best source of serious demo goals because it does not ask an LLM to invent random lemmas. It imports Mathlib, walks Lean's elaborated environment, filters declarations under CategoryTheory, pretty-prints theorem types, checks that statements are usable, and runs the local tactic ladder.



Result labels:

Label	Meaning	Demo interpretation
solved locally	Cheap local tactics found a proof.	Good sanity check; not worth Aristotle.
interesting miss	Statement is well-formed and known in Mathlib, but local tactics missed.	Good proof-reconstruction candidate.
unusable	The rendered/generated statement did not parse locally.	Do not submit; fix formalization first.

Candidate scoring uses local heuristics: topic, namespace signal, statement length, explicit elaborator noise, and whether the statement parsed. A score of 0 usually means Lean cannot even read the statement as a proposition.

6. Agentic Theorem Proposal Chain

The app has a deliberately small, gated agentic chain. It can use local templates or a cheap OpenAI/Anthropic proposer. The model only proposes prose; Aristotle handles formalization; Lean remains the checker.



- This is agentic because it chooses a candidate from state and records artifact transitions.
- It is not autonomous: Aristotle spend requires explicit clicks.
- It is not LangGraph: the chain is small enough to express as ordinary Python and Lean state machines.
- A generated statement can be true and Lean-provable, but novelty still requires collision search and expert review.

7. Copresheaf Artifact Model

The file `Formalize/PaperBridge.lean` gives a static categorical bridge for artifact/provenance flow. It builds a schema category as the free category on a quiver of artifact transitions, then treats an artifact state as a functor from that schema into `Type`.



ArtifactOp edges: `checkWellFormed` and `runOracle`. Mathlib Paths supplies the free category.

Lean shape:

```
abbrev Schema := Paths ArtifactType
abbrev ArtifactState := Schema -> Type
-- In Lean source this is written with the category-theory functor arrow.
def initialState : ArtifactState := Paths.lift initialStateProto
abbrev Provenance (I : ArtifactState) := I.Elements
```

Important caveat: This is currently static. The live miner/proposer/oracle still run as Lean/Python effects and write JSON/archive artifacts. The copresheaf bridge models the typed shape of artifact flow; it does not hide network calls inside a pure Lean functor.

8. Aristotle Integration

Aristotle command	Role in the project	Trust status
formalize	Turns prose/LaTeX into a Lean statement.	Untrusted; local parser only checks shape.
submit	Fills a sorry in a Lean project.	Untrusted until local oracle re-verifies.
download/status	Fetches result summaries and Lean files.	Evidence source, not final authority.

Tests and replay fixtures are offline by construction. Live Aristotle calls happen only from explicit UI actions or explicit commands. This is both a cost-control measure and a reproducibility discipline.

9. Category Theory Concepts to Be Comfortable With

Category	A collection of objects and morphisms with identities and associative composition.
Functor	A structure-preserving map between categories. It sends objects to objects and morphisms to morphisms while preserving identity and composition.
Natural transformation	A morphism between functors, with one component morphism at each object, satisfying a compatibility square.
Adjunction	A pair of functors that are related by a natural hom-set correspondence. Left adjoints commonly preserve colimits; right adjoints preserve limits.
Limit/colimit	Universal ways to assemble or summarize diagrams. Products, pullbacks, coproducts, and coequalizers are examples.
Yoneda	A principle saying objects are understood through maps into or out of them. Useful for turning universal properties into representable functor statements.
Copresheaf	A functor from a small category into Type or Set. In this project it indexes artifact data by schema positions.
Category of elements	Given a functor $I : C \rightarrow \text{Type}$, its objects are pairs (c, x) where x is an element of $I(c)$. Here it acts as a provenance category.

Do not pretend to be a category theorist beyond the project. A strong answer is: I use these concepts to structure the interface, artifact flow, and proof pipeline; I am still learning the deeper mathematics.

10. Likely Interview Questions

Q: Why move from Idris to Lean?

Lean has Mathlib, Aristotle compatibility, and a kernel suitable for local verification of real category theory. Idris was useful for the initial dependent-container idea, but Lean is the right ecosystem for proof reconstruction.

Q: What is genuinely categorical here?

The MCP interface is represented polynomially; the agent/server interaction is lens composition; artifact state is modeled as a copresheaf over a free schema category.

Q: What is agentic AI here?

A small gated chain proposes theorem candidates from a mined seed, records artifacts, asks Aristotle to formalize/prove only after explicit clicks, and verifies locally. It is not an autonomous discovery engine.

Q: Could this discover new category theory?

A generated candidate can be true and proved in Lean, but novelty is not established. The current project can produce verified candidates; novelty needs semantic Mathlib search, literature search, and expert review.

Q: Why not trust Aristotle?

Because it is an external prover/service. Its output is useful only after local Lean elaboration, axiom audit, and statement matching.

Q: What is an interesting miss?

A real Mathlib theorem whose existing proof is hidden and whose statement the local tactic ladder cannot reprove cheaply. It is a proof-reconstruction task, not a new theorem.

Q: Why not use LangGraph?

The chain is mostly linear and human-gated. Ordinary Python plus a Lean state machine is simpler and easier to audit. A graph runtime would help only for branching, parallel, resumable multi-agent exploration.

11. Demo Script

- Open the local UI at `http://localhost:8770`.
- Click **Mine statement**. Explain that the statement is from real Mathlib CategoryTheory, not generated by an LLM.
- Point to the local result: solved, interesting miss, or unusable.
- Click **Find Aristotle candidate**. Explain candidate scoring and why bad candidates are skipped.
- Click **Propose theorem**. Explain the gated agentic chain: open need, seed context, proposer, gate.
- If available, show a formalized proposition. If it is unusable, say plainly: Lean could not parse it, so sending it for proof would waste credits.
- Show the README or code path for the trust model: Lean verification, axiom whitelist, statement match.

Useful command-line checks:

```
lake build
./scripts/check.sh
./scripts/check.sh --full
lake exe miner-report --limit 2 --max-tier 0 --json
```

12. Limitations and Next Steps

Current limitation	Best next step
Generated theorem may be standard or already in Mathlib.	Add semantic/corpus search over formalized statements and nearby declarations.
Formalization can return prose or invalid Lean.	Add stricter formalization prompts, parser diagnostics, and automated repair loops.
Copresheaf bridge is static.	Make the live JSON artifact store conform to the schema and export category-of-elements records.
Local tactics are intentionally small.	Add richer tactic tiers while keeping axiom and statement checks unchanged.
No claim of mathematical novelty.	Add human review, literature search, and a novelty report for verified generated statements.

Closing line to remember: The project is a trustworthy pipeline for generating, mining, formalizing, attempting, and locally verifying category-theory proof tasks. Its strength is the trust boundary, not hype about autonomous discovery.